

Moderní vyvoj s AI

Průvodce context engineeringem a tvorbou .md souborů

Aktualizováno: kveten 2026 · Zdroje: martinfowler.com, Packmind, ETH Zurich, Cortex, builder.io, agents.md

1. Proč záleží na kontextu

AI nástroje pro vývoj jsou v roce 2026 standardem – používá je 91 % inženýrských organizací. Rychlost generování kódu vzrostla, ale s ní i technický dluh: bez řízeného kontextu AI generuje kód konzistentní se svým tréninkem, ne s vaším projektem.

Zlaté pravidlo: AI ví **jak** psát kód – ale bez kontextu neví **proč** a **co**. .md soubory jsou most mezi oběma světy.

Data z praxe (2025-2026)

Zdroj	Zjištění
Cortex, 2026	Týmy bez řízeného kontextu: +20 % PR, +23,5 % incidentů, +30 % change failure rate
ETH Zurich, 2026	LLM-generované context soubory snižují úspěšnost o 3 %; ručně psané přidávají +4 %
GitClear, 2024-25	8× nárůst duplicitních bloků kódu; podíl refaktoringu klesl z 24 % na pod 10 %
ACE / Stanford, 2025	Strukturované context updates snižují drift a latenci až o 86 %
Cortex, 2026	Jen 32 % organizací má formální AI governance – přestože 91 % nástroje aktivně používá

2. Soubory a hierarchie kontextu

Každý AI nástroj čte jiný soubor. Moderní přístup: jeden kanonický zdroj pravdy, ze kterého se generují tool-specifické formáty automaticky.

Soubory podle nástroje

Soubor	Nástroj	Umístění
CLAUDE.md	Claude Code	root / modul
AGENTS.md	Univerzální standard (agents.md, 2026)	root / modul
.cursor/rules/*.mdc	Cursor – scoped rules	root projektu

.github/copilot-instructions.md	GitHub Copilot	.github/
.kiro/steering/	Kiro (AWS)	root projektu

Doporučená hierarchie v projektu

CLAUDE.md / AGENTS.md <- root: přehled projektu, tech stack (max 1-2 strany)
src/
core/CLAUDE.md <- kontext pro core modul
appblock/CLAUDE.md <- kontext pro blokovani (viz Priloha A)
statistics/CLAUDE.md <- kontext pro statistiky
widgets/CLAUDE.md <- kontext pro widgety

Agent načte jen kontext relevantní pro aktuální pracovní adresář. Root soubor se načítá vždy, modulové soubory jen při práci v daném modulu.

3. Co patří do .md a co do komentářů

Oba přístupy jsou komplementární, ne konkurenční. Klíč je správné přiřazení obsahu.

.md soubory - systémová úroveň	Komentáře v kódu - implementační úroveň
High-level přehled a účel modulu	Jak konkrétní funkce funguje
Proč bylo architektonické rozhodnutí přijato	KDoc / Javadoc pro veřejné API
Vztahy a hranice mezi moduly	Kontext přímo u netriviačního kódu
Varování před Android-specifickými pastmi	Odkaz na issue / ticket
Business pravidla a výjimky	Edge cases konkrétní implementace
Explicitní anti-patterns (co NEDĚLAT)	TODO / FIXME s kontextem

4. Co má dobrý AI kontext obsahovat

Výzkum Birgitta Böckeler (Thoughtworks / martinfowler.com, únor 2026) identifikoval strukturu, která konzistentně produkuje nejlepší výsledky:

Vrstva	Obsah	Priorita
1. Přehled projektu	Co aplikace dělá, tech stack, klíčové závislosti	Kritická
2. Architektura	Proč MVI, přímý DAO přístup, hranice modulů	Kritická
3. Příkazy	Build, test, lint - agenti je přímo spouštějí	Vysoká
4. Anti-patterns	Explicitní zákazy: runBlocking, !!, GlobalScope...	Vysoká

5. Domain knowledge	Projektová terminologie, zkratky, entity	Střední
6. Ukázky dobrého kódu	Odkaz na konkrétní soubory jako vzor	Střední

Zahrnout ✓

- ✓ Explicitní anti-patterns – dle výzkumů nejefektivnější část celého souboru
- ✓ Projektová terminologie a zkratky – AI pak rozumí 'profil blokování' bez vysvětlování
- ✓ Příkazy pro build a testy – agent si je může rovnou spustit a validovat svou práci
- ✓ Odkaz na konkrétní soubory jako vzor: *prefer functional components like Projects.tsx*
- ✓ Varování před platformními pastmi (výrobci, verze OS, permissions)

Vynechat ✗

- ✗ AI-generovaný obsah bez ruční revize – ETH Zurich (2026): snižuje úspěšnost o 3 %
- ✗ Obecné principy (DRY, SOLID, Clean Code) bez projektového kontextu – AI je zná
- ✗ Architektonický přehled struktury složek – modely si ho umejí načíst samy
- ✗ Příliš dlouhé sekce – 'lost in the middle' efekt, agent dělá víc kroků bez lepšího výsledku
- ✗ Zastaralý obsah – horší než žádný kontext (viz sekce 6: context drift)

5. Velikost souborů a token budget

Více kontextu není vždy lepší. Birgitta Böckeler (martinfowler.com, únor 2026) dokumentuje kontraintuitivní zjištění: příliš mnoho kontextu snižuje efektivitu agenta. Klíčový princip: do CLAUDE.md patří jen to, co agent potřebuje vědět **před tím, než začne pracovat**. Vše ostatní linkujte do samostatných souborů.

Soubor	Co obsahuje	Co NEobsahuje - linkovat externě
Root CLAUDE.md	Project overview, tech stack, globální anti-patterns, build příkazy. Max čitelný za 2 minuty.	Detaily jednotlivých modulů, tabulky tříd, diagramy
Modulový CLAUDE.md	Účel modulu, klíčové business pravidla, platformní pasti, modulární anti-patterns.	Séznamové tabulky všech tříd, podrobné diagramy, ADR → linkovat do ARCHITECTURE.md apod.
Cursor .mdc	Konkrétní pravidlo pro daný typ souboru (glob)	Vše co se netvýká glob patternu souboru
Propojené docs	Tabulky, diagramy, ADR, OEM poznámky – libovolná délka	-

Token budget jako quality gate: Nastavte maximální velikost CLAUDE.md a vynucujte ji v CI/CD – stejně jako lint. Překročení = signál, že něco patří do propojeného souboru.

6. Context drift - největší riziko roku 2026

Context drift nastává, když .md soubory popisují pravidla, která tým už dávno nepoužívá. AI generuje kód, který vypadá správně, ale systematicky porušuje aktuální standardy. Chyba není viditelná v jednom PR – kumuluje se jako technický dluh.

Klasický příklad: CLAUDE.md stále říká 'používáme Jest' – tým přešel na Vitest před třemi měsíci. Každý AI-generovaný test je od té doby špatně. Dnádny build error, jen systematická chyba v každém PR.

Jak předcházet context driftu

- ▶ .md soubory jsou součástí **PR review** stejně jako kód – změna frameworku = povinná aktualizace souboru
- ▶ Když AI udělá předpoklad, který chceš opravit: **řekni jí, ať to přidá do CLAUDE.md** – trvalá oprava místo jednorázové korekce
- ▶ Preferovat **evergreen pravidla** (proč, ne jak) – méně zastarávají při refaktoringu
- ▶ Pravidelný audit: číst context soubory pohledem AI agenta – je každá instrukce stále spustitelná a platná?
- ▶ Nástroje jako **Packmind context-evaluator** automaticky detekují zastaralý obsah
- ▶ Datum poslední revize v hlavičce souboru – okamžitý signál, zda je kontext čerstvý

7. Context engineering vs. prompt engineering

V roce 2026 se ustálil pojmový rozdíl, který má zásadní dopad na to, jak o kontextu uvažovat (Martin Fowler, martinfowler.com, únor 2026):

Prompt engineering	Context engineering
Platí jen pro jednu konverzaci	Platí pro celý tým a všechny nástroje
Investice zmizí po zavření chatu	Commitnutá do repozitáře, přetrvává
Osobní produktivita jednotlivce	Organizační infrastruktura
Konverzační dovednost	Systémová disciplína
Musí se opakovat při každém sezení	Jedno nastavení, neomezené použití

"Prompt engineering is a conversation skill. Context engineering is a systems discipline." – Packmind, 2026

8. ContextOps - context engineering na úrovni organizace

Gartner identifikoval context engineering jako kritickou DevOps kompetenci pro rok 2026. ContextOps přináší DevOps logiku do správy AI kontextu: tak jako CI/CD zprůmyslnilo deployment, ContextOps zprůmyslňuje tvorbu, distribuci a správu kontextových souborů.

Čtyři fáze ContextOps cyklu

Fáze	Co se děje	Nástroje
1. Build	Skenovani repo, detekce vzorů, generování standardů z kódu	Packmind /init, Claude Code /init
2. Distribute	Jeden playbook → CLAUDE.md, .cursor/rules, copilot-instructions.md najednou	Packmind, vlastní skripty
3. Govern	Verzování, PR review pro .md změny, enforcement v CI/CD pipeline	Git, Packmind linter
4. Maintain	Centralizované updaty propagované do všech repo, detekce driftu	context-evaluator, Packmind

Klíčové trendy pro rok 2026

- ▶ **AGENTS.md jako univerzální standard** – prosazuje se jako tool-agnostický formát čtený všemi agenty
- ▶ **AI-asistovaná aktualizace** – každá oprava: 'přidej to do CLAUDE.md' místo jednorázové korekce
- ▶ **Multi-agent governance** – Gartner: 1 445 % nárůst dotazů na multi-agent systémy (Q1 2024 – Q2 2025)
- ▶ **/init jako startovní bod, ne konečný výsledek** – draft je nutné ručně dopracovat o business kontext
- ▶ **Context soubory v CI/CD** – token budget jako quality gate, lint na zastaralé reference

9. Šablona root souboru

Následující struktura je ověřená komunitou a odpovídá oficiální Anthropic dokumentaci. Každá sekce má jasný účel – pokud sekce nepomáhá agentovi udělat lepší změnu v kódu, nepatří sem.

```
# Project Context

# Owner: | Last reviewed:

## What this project is

One paragraph: what the product does and who uses it.

Write it as if briefing a contractor with zero prior context.

## Tech stack

Languages, frameworks, build system, test tools – with version constraints that matter.

Example: Kotlin 2.x, Jetpack Compose, Room, Hilt, MVI (not MVVM)

## Architecture principles
```

Key decisions and WHY they were made.

Example: Direct DAO access via UseCases – no Repository layer unless multiple sources.

Commands

`./gradlew test` <- run before every PR

`./gradlew lint` <- required to pass CI

`./gradlew assembleDebug` <- build

Global rules

Rules that apply across all modules. Keep this list short.

Example: Never use `runBlocking` in production. Use `CrashHelper.logException` for exceptions.

What NOT to do

Explicit anti-patterns – this section has the highest ROI.

Example: Do not use `!!` operator. Do not expose suspend functions from `ViewModel` to UI.

Good examples

Links to specific files as canonical patterns – not generic principles.

Example: Follow `BlockRuleEvaluator.kt` for pure business logic without Android deps.

Linked docs

Links to longer docs agents load on demand (ADR, diagrams, OEM notes).

`@docs/architecture.md` <- module relations and data flow

`@docs/oem-notes.md` <- Android OEM pitfalls per manufacturer

Klíčový test každého řádku (Anthropic, oficiální dokumentace): "Udělal by agent chybu bez této instrukce?" Pokud ne – smaz ji. Přebytné instrukce rozedřibají ty důležité.

10. Dobré vs. špatné pravidlo

Nejcennější dovednost při tvorbě context souborů je psát pravidla, která AI skutečně použije. Rozdíl mezi obecnou instrukcí a aktní instrukcí je zásadní.

ŠPATNĚ: příliš obecné Write clean code.	DOBŘE: konkrétní a aktní Never use <code>runBlocking</code> in Android Services or ViewModels. It causes ANR. Use <code>suspend</code> functions with <code>Dispatchers.IO</code> instead.
ŠPATNĚ: AI to už ví Follow SOLID principles.	DOBŘE: projektový kontext Direct DAO access via UseCases only. Do not add a Repository layer unless coordinating multiple data sources (see architecture.md).
ŠPATNĚ: línk na dokumentaci See our coding standards at wiki/standards.	DOBŘE: přímé pravidlo Do not pass <code>packageName</code> as plain String across module boundaries. Use the value class <code>AppPackage</code> to prevent mixing up identifiers.

11. Praktický checklist a rollout

Checklist pro vývojáře

- ✓ Měním architekturu nebo build systém? → aktualizuji root AGENTS.md / CLAUDE.md
- ✓ Měním chování modulu? → aktualizuji modulový context soubor
- ✓ AI udělala špatný předpoklad? → doplním pravidlo do kontextu místo jednorázové opravy v chatu
- ✓ Context soubor je delší než domluvený limit? → přesouvám detail do propojeného souboru
- ✓ Každý context soubor má ownera a datum poslední revize v hlavičce
- ✓ Do PR šablony patří otázka: **"Mění tato změna pravidla, která má znát AI agent?"**
- ✓ Jednou za kvartál: krátký audit – čteme context soubory pohledem agenta, mazáme zastaralé části

Doporučený rollout ve fázích

Klíčový princip: nezačínat celým projektem. Vybrat 2–3 nejkritičtější moduly, ověřit hodnotu a pak rozšířit.

Fáze	Co udělat	Kritérium dokončení
1. Základ	Root AGENTS.md: přehled produktu, tech stack, build/test příkazy, globální anti-patterns	AI generuje kód konzistentní s tech stackem bez opravy
2. Křičtičké moduly	Modulové context soubory pro 2–3 nejrizikovější části (blocking, paywall, onboarding)	Pokles opakovaných korekcí v PR review týkajících se těchto modulů
3. CI/CD	Limit velikosti context souborů jako quality gate, otázka do PR šablony	Drift je zachycen před merge, ne až v kódu

4. Rozšíření	Zbylé moduly, kvartální audit, pravidla z chyb AI	Context soubory se aktivně aktualizují při každé architektonické změně
-----------------	---	--

Pokračování: Příloha A – Modelový příklad CLAUDE.md pro Android modul blokování →

Note: This is a model example only. Class names, module structure, and Android-specific details are illustrative. The example shows the recommended structure and content depth – not actual AppBlock source code. A real CLAUDE.md would be written by developers who know the codebase.

Note on length: This example is intentionally longer than the recommended 0.5–1 page because it shows all possible sections in one place. In a real project, most sections would be split into linked files: OEM pitfalls → OEM_NOTES.md, class table → ARCHITECTURE.md. The actual CLAUDE.md would keep only: module purpose, detection flow summary, business rules, and What NOT to do – fitting comfortably on one page.

AppBlock - Blocking Module

```
# AppBlock - Blocking Module

# Last updated: May 2026

# Scope: src/appblock/ - read by Claude Code, Cursor, and compatible agents
```

What this module does

The **appblock** module detects and blocks foreground apps. It combines two detection methods because neither works reliably across all Android OEMs:

Method	Role	Always active
AccessibilityService	Primary - real-time TYPE_WINDOW_STATE_CHANGED events	Yes
UsageStatsManager	Secondary - polling every 500 ms on Dispatchers.IO	Yes (fallback)

Both methods run in parallel. **AppDetectionCoordinator** merges results and decides whether to show the block screen.

Key classes

Class	Responsibility
BlockAccessibilityService	Android Service - listens for window change events
UsageStatsPoller	Coroutine loop - polls UsageStats every 500 ms
AppDetectionCoordinator	Merges both detectors, throttling, blocking decision. Canonical code example - follow this pattern.

BlockScreenActivity	Shows block screen via WindowManager overlay, extends BaseComposeActivity
BlockRuleEvaluator	Pure business logic – no Android deps, fully unit-testable. Best starting point for new rules.
ActiveProfileProvider	Exposes the active blocking profile as StateFlow

Detection flow

```

App in foreground
|
+-- AccessibilityService -----+
| v
+-- UsageStatsPoller ----> AppDetectionCoordinator
|
BlockRuleEvaluator
(is app blocked?)
|
+-----+-----+
NO YES
| |
do nothing BlockScreenActivity
(WindowManager overlay)

```

Business rules

- ✓ Blocking is **never** applied to system apps (isSystemApp == true)
- ✓ Blocking is **never** applied to AppBlock itself – prevents user lockout
- ✓ **Grace period:** 3-second delay after detection before showing block screen (avoids false positives during fast switching)
- ✓ **Emergency unlock:** user enters PIN for temporary unblock (default: 5 minutes)
- ✓ If the device is on the lock screen, blocking is skipped entirely
- ✓ **Profiles:** each profile contains blocked apps list, a time window, and days of week

Android OEM pitfalls – read before making changes

This section contains hard-won knowledge. Skipping it will cause regressions on specific devices.

Xiaomi / MIUI

- ▶ WindowManager overlay does not work unless BlockAccessibilityService runs as a **foreground service**
- ▶ MIUI battery saver aggressively kills background services – service must periodically renew its foreground notification
- ▶ Workarounds are in MiuiCompat.kt – do not inline them elsewhere

Samsung / One UI

- ▶ TYPE_WINDOW_STATE_CHANGED events occasionally missing when transitioning between Samsung native activities
- ▶ UsageStatsPoller is therefore critical – do not remove or disable even if AccessibilityService seems sufficient

Android 12+

- ▶ UsageStatsManager requires PACKAGE_USAGE_STATS – user must grant it manually in system settings
- ▶ If permission missing: fall back to AccessibilityService only and show a prominent warning
- ▶ Always check Settings.canDrawOverlays(context) before any WindowManager overlay attempt

What NOT to do

- ✗ **Never use runBlocking** in BlockAccessibilityService or UsageStatsPoller – causes ANR
- ✗ **Never call UsageStats on the main thread** – blocking I/O, always use withContext(Dispatchers.IO)
- ✗ **Never add WindowManager overlay without checking permission** – crashes on Android 12+
- ✗ **Never use GlobalScope** for the polling loop – lifecycle not bound to service, causes leaks
- ✗ **Do not rely solely on AccessibilityService** – see Samsung section above
- ✗ **Do not pass packageName as plain String** across layers – use value class AppPackage
- ✗ **Do not add direct module dependencies** – communicate with statistics and widgets via Flow only

Testing

```
test/  
  
BlockRuleEvaluatorTest.kt <- unit tests, no Android deps, cover all edge cases  
  
AppDetectionCoordinatorTest.kt <- mock both detectors, test merge logic  
  
androidTest/  
  
BlockAccessibilityServiceTest.kt <- instrumented, requires real Android context  
  
UsageStatsPollerTest.kt <- instrumented
```

BlockRuleEvaluator is the highest-value test target – pure logic, fast, covers all business rules. Aim for 100 % branch coverage here.

Module relations

Module	Relationship	Direction
core	Provides BaseStatefulViewModel, CrashHelper, base classes	appblock depends on core
statistics	Consumes detection events via Flow - no direct calls	statistics listens to appblock
widgets	Reads ActiveProfileProvider - read-only, no writes	widgets reads appblock